

Wind Turbine EDA Project

1 Wind Turbine EDA Project

🔖 Python: Project Overview

A comprehensive data analysis project for a renewable energy private equity firm. The goal was to identify the most successful U.S. wind farm blueprints and recommend acquisition targets.

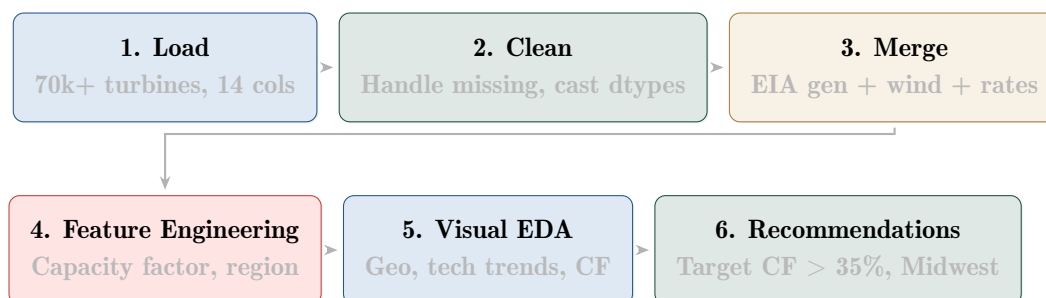
Datasets used:

- **U.S. Wind Turbine Database** — 70,000+ turbines with specs (capacity, hub height, rotor diameter, year installed, location, manufacturer)
- **EIA-923 Power Plant Report (2025)** — *actual* net generation (MWh) for every U.S. power plant, full year
- **State-level wind speeds** — average wind speed (mph) by state
- **State electricity rates** — commercial & residential cents/kWh

Engineering target: $\text{Capacity_Factor} = \text{Net_Generation_MWh} / (\text{t_cap} \times 8760)$ — the single metric that tells you whether a turbine is *actually* performing.

1.1 Analysis Pipeline

The project followed a structured 6-stage EDA pipeline:



1.2 Stage 1—2: Data Loading and Cleaning

The US Wind Turbine Database required significant cleaning before analysis could begin:

Inspect shape and dtypes:

```

1 import pandas as pd
2 import numpy as np
3
4 df = pd.read_csv('data/us-wind-turbines.csv')
5
6 # Inspect shape and dtypes

```

Key cleaning steps::

```

1 print(df.shape)           # (70000+, 14 columns)
2 print(df.info())
3 print(df.isna().sum())
4
5 # Key cleaning steps:
6 # 1. Handle missing p_year (project year) --- drop or impute

```

2. Cast `t_cap` (capacity) to numeric, handle anomalies:

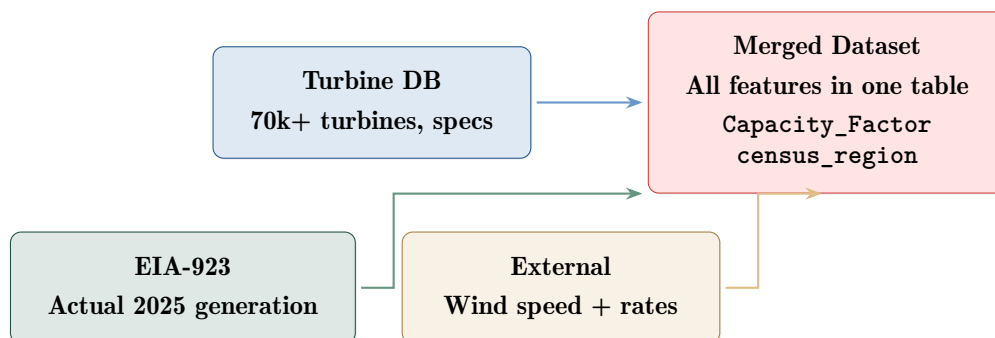
```
1 # 2. Cast t_cap (capacity) to numeric, handle anomalies
2 # 3. Standardise column names (lowercase, underscores)
3 # 4. Drop duplicate case_id entries
4 # 5. Handle t_state: some entries are invalid/empty
5 # 6. Retrofit column: convert to boolean (0 = no, 1 = yes)
```

Python: Data Dictionary (Engineered)

Column	Type	Description
case_id	int	Unique turbine ID
t_state	str	State (2-letter code)
p_year	int	Year operational
t_cap	float	Nameplate capacity (kW)
t_hh	float	Hub height (m)
t_rd	float	Rotor diameter (m)
retrofit	int	1 = retrofitted
Net_Generation_MWh	float	Actual 2025 generation
WindiestStatesAverageWindSpeedMPH	float	State avg wind speed
Average (elec rate)	float	State electricity rate
Capacity_Factor	float	Engineered: NetGen / (t_cap × 8760)
census_region	str	Engineered: region from t_state

1.3 Stage 3—4: Merging and Feature Engineering

The real analytical power came from **combining four separate data sources**:



The engineered **Capacity Factor** is what transforms raw data into actionable insight. It answers: “Given this turbine’s nameplate rating, how much of its potential did it *actually* deliver?”

$$\text{Capacity Factor} = \frac{\text{Net_Generation_MWh}}{\text{t_cap_MW} \times 8760} \quad \text{where} \quad \text{t_cap_MW} = \frac{\text{t_cap_kW}}{1000}$$

Merge generation data by matching plant names/IDs:

```
1 # Merge generation data by matching plant names/IDs
2 df = pd.merge(turbine_df, eia_gen_df[['Plant_Name',
3   Net_Generation_MWh']],
4               on='Plant_Name', how='inner')
```

```

5 # Merge state-level wind speed and electricity rates
6 df = pd.merge(df, wind_speed_df, on='t_state', how='left')
7 df = pd.merge(df, elec_rates_df, on='t_state', how='left')

```

Feature engineer: Capacity Factor (convert t_cap from kW to MW):

```

1
2 # Feature engineer: Capacity Factor (convert t_cap from kW to MW)
3 df['Capacity_Factor'] = df['Net_Generation_MWh'] / \
4     ((df['t_cap'] / 1000) * 8760)
5
6 # Feature engineer: Census Region from state codes
7 state_to_region = {

```

Map state codes to census regions:

```

1     'IA': 'Midwest', 'IL': 'Midwest', 'MN': 'Midwest', 'KS': 'Midwest',
2     'TX': 'South', 'CA': 'West', 'NY': 'Northeast', # ... all 50
3     states
4 }
5 df['census_region'] = df['t_state'].map(state_to_region)

```

1.4 Stage 5: Key Analytical Findings

1.4.1 Geographic Concentration

The first clear pattern: wind energy is **highly concentrated**. The Midwest + Texas alone hold roughly **55% of all active U.S. turbines**.

```

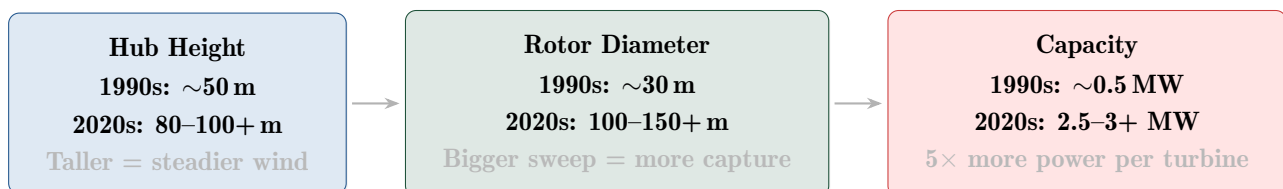
1 # Geographic concentration
2 region_counts = df['census_region'].value_counts(normalize=True)
3 * 100
4 print(region_counts)

```

This concentration matters because it means grid infrastructure, maintenance expertise, and supply chains are already established in these regions — lowering acquisition risk.

1.4.2 Technological Shift Over Time

The most dramatic finding: turbines installed **after 2015** produce **5× more power** than those from the 1990s. Two engineering drivers explain this:



```

1 # Average capacity by decade
2 df['decade'] = (df['p_year'] // 10) * 10
3 decade_stats = df.groupby('decade')['t_cap'].mean()
4 # 1990s: ~500 kW -> 2020s: ~2500+ kW (5x increase!)

```

1.4.3 Capacity Factor by State

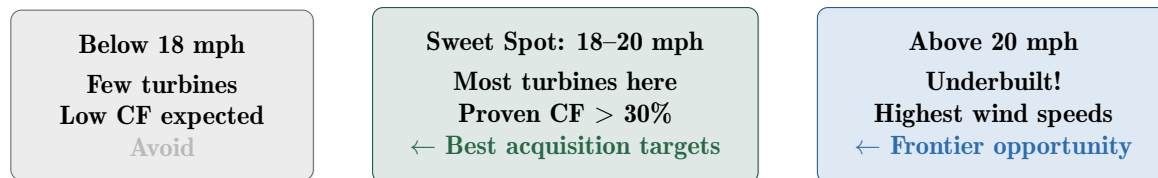
When actual generation data is factored in, the performance picture shifts dramatically:

```
1 # Top CF states
2 state_cf = (df.groupby('t_state')['Capacity_Factor']
3             .mean().sort_values(ascending=False))
4 print(state_cf.head(10))
5 # New Mexico, Wyoming > 35% -- These are the blueprints to
   follow
```

New Mexico and **Wyoming** exceeded 35% capacity factor — well above the national average. These are the proven blueprints the firm should target.

1.4.4 Wind Speed Sweet Spot

Cross-referencing turbine locations with state-level wind speeds revealed a counter-intuitive pattern:



Key insight: States with the absolute highest wind speeds (20+ mph) remain surprisingly **underbuilt** — very few turbines installed. The industry has clustered in the 18–20 mph range, not at the extremes. This creates a second-mover opportunity: build modern turbines in high-wind, underbuilt states.

1.4.5 Manufacturer Market Share

The turbine manufacturing market is highly concentrated. GE Wind dominates with over 40% market share, while Nordex produces the highest-average-capacity turbines.

```
1 # Manufacturer distribution
2 manu_counts = df['t_manu'].value_counts(normalize=True) * 100
3 print(manu_counts.head(5))
4 # GE Wind:      41.1%
5 # Vestas:      18.5%
6 # Siemens:     10.2%
7 # Mitsubishi:   6.8%
8 # Nordex:       4.3%
9
10 # Avg capacity by manufacturer
11 manu_cap = df.groupby('t_manu')['t_cap'].mean().sort_values(
12               ascending=False)
13 print(manu_cap.head(3))
14 # Nordex:      3,195 kW
15 # GE Wind:     2,780 kW
16 # Vestas:      2,510 kW
```

Regional patterns also emerge: GE Wind leads in the Midwest and South, while Mitsubishi is concentrated in the South and Vestas has a strong West Coast presence.

1.4.6 Retrofit Analysis

8.5% of the fleet (5,986 turbines) have been retrofitted — their original turbines replaced with modern, more efficient ones while reusing the same land and grid connection. This is an important acquisition strategy.

```

1 # Retrofit statistics
2 retrofit_count = df['retrofit'].sum()
3 print(f"Retrofitted turbines: {retrofit_count:,}")
4
5 retrofit_by_state = df[df['retrofit'] == 1]['t_state'].
   value_counts().head(5)
6 # Texas:    2,539
7 # Iowa:     1,179
8 # ...

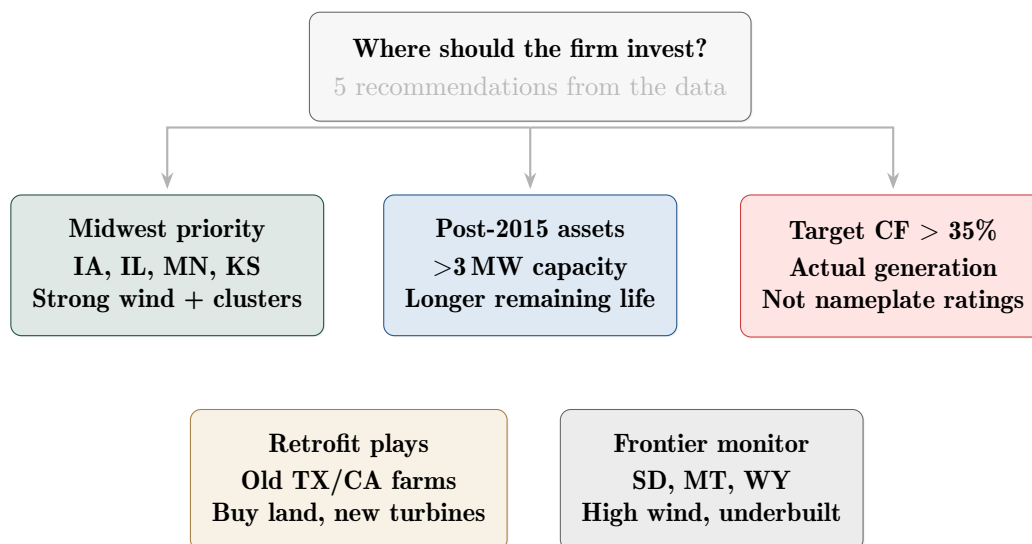
```

Most retrofits occurred between 2015 and 2020. Texas leads with 2,539 retrofitted turbines — the state's excellent wind resources make upgrading old turbines highly profitable.

1.4.7 Nameplate vs Reality

1.5 Stage 6: Investment Recommendations

The analysis converges on five concrete recommendations:



1. **Prioritise the Midwest:** Iowa, Illinois, Minnesota, Kansas — strong wind + dense clusters + competitive electricity rates, proven infrastructure
2. **Focus on post-2015 assets:** Capacity > 3 MW, longer remaining life, modern engineering (taller towers, larger rotors)
3. **Use capacity factors, not nameplates:** Target > 35% CF — verified through actual EIA generation data, not manufacturer specs
4. **Look for retrofit plays:** Old Texas/California farms with excellent wind but obsolete turbines — buy the land and grid connection, replace the turbines
5. **Monitor the frontier:** South Dakota, Montana, Wyoming — highest wind speeds in the country (20+ mph), still dramatically underbuilt, first-mover opportunity

🐍 Python: Why This Project Matters

The Wind Turbine EDA demonstrates the **full data science workflow**: messy real-world data → cleaning → merging multiple sources → feature engineering → visual EDA → actionable recommendations. It's not just about making charts — it's about making **investment decisions** backed by evidence.

1.6 Limitations and Further Research

⚠ Warning: Data Caveats

Key limitations to consider when interpreting these results:

- **EIA-923 data status:** The 2025 dataset used was an *Early Release* — final figures may differ slightly.
- **Unmatched turbines:** Approximately 11% of turbines could not be matched to EIA generation data and were excluded from capacity factor analysis.
- **Electricity price data:** State-level averages mask significant regional variation within states.
- **Wind speed data:** State-level averages were used; actual site-level wind speeds would provide more precise siting analysis.
- **No maintenance data:** Downtime and maintenance records would explain some of the nameplate–capacity–factor discrepancies.
- **Transmission costs:** Grid connection and transmission costs were not included in the financial analysis.

💡 Tip: Data Sources

- **U.S. Wind Turbine Database:** <https://eerscmap.usgs.gov/uswtdb>
- **EIA-923 Power Plant Report:** <https://www.eia.gov/electricity/data/eia923>
- **Wind speed data:** NOAA / NREL wind resource maps
- **Electricity rates:** EIA Annual Electric Power Industry Report